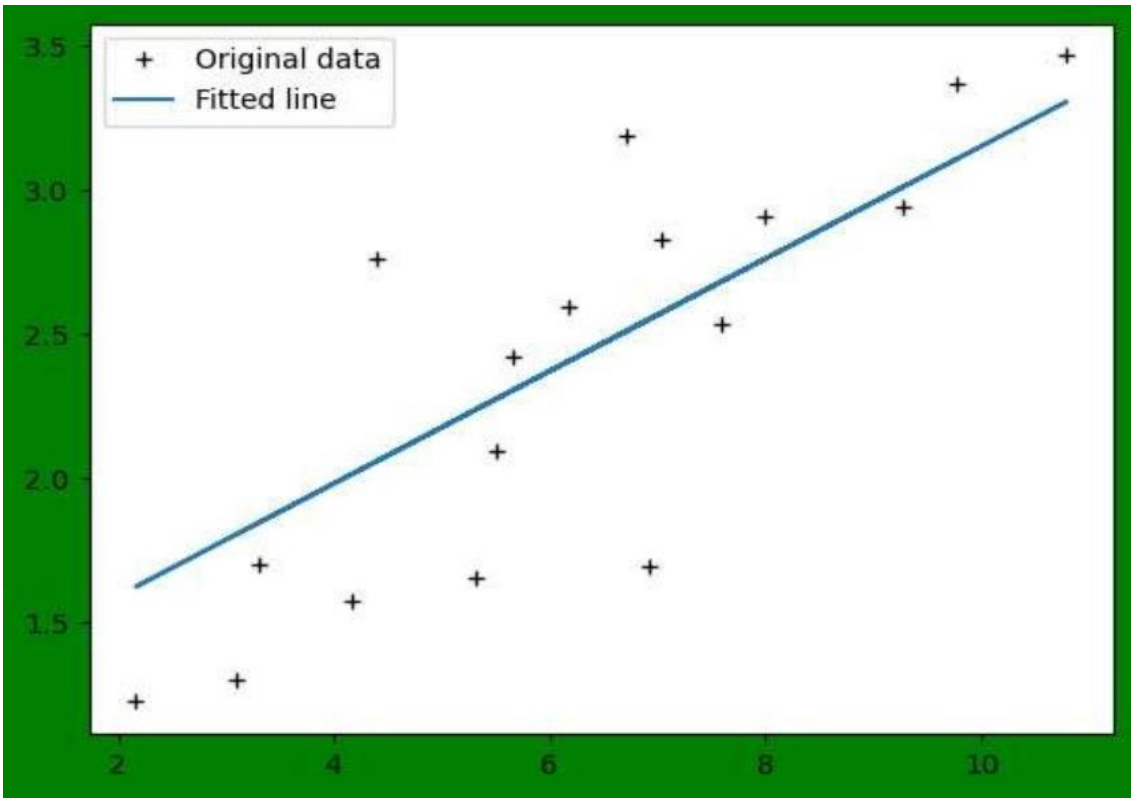
SID : 2534321

LAB1



LAB 2 :

```
Test Accuracy: 1.000
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	350
dense_1 (Dense)	(None, 67)	737
dense_2 (Dense)	(None, 56)	3,808
dense_3 (Dense)	(None, 34)	1,938
dense_4 (Dense)	(None, 1)	35

Total params: 20,606 (80.50 KB)

Trainable params: 6,868 (26.83 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 13,738 (53.67 KB)

LAB : 3

Test accuracy: 0.8736000061035156

```
[27]: print(model.summary())
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 56)	43,960
dense_1 (Dense)	(None, 45)	2,565
dense_2 (Dense)	(None, 32)	1,472
dense_3 (Dense)	(None, 24)	792
dense_4 (Dense)	(None, 10)	250

Total params: 49,039 (191.56 KB)

Trainable params: 49,039 (191.56 KB)

Non-trainable params: 0 (0.00 B)

None

LAB : 4

```
model = Sequential()

# Your code to add as many layers with activation functions of your choice:

model.add(Dense(200,activation='relu',input_shape=(n_features,)))
model.add(Dense(150,activation='relu',input_shape=(n_features,)))
model.add(Dense(100,activation='relu',input_shape=(n_features,)))
model.add(Dense(80,activation='relu',input_shape=(n_features,)))
model.add(Dense(40,activation='relu',input_shape=(n_features,)))


model.add(Dense(1))
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 200)	4,000
dense_1 (Dense)	(None, 150)	30,150
dense_2 (Dense)	(None, 100)	15,100
dense_3 (Dense)	(None, 80)	8,080
dense_4 (Dense)	(None, 40)	3,240
dense_5 (Dense)	(None, 1)	41

Total params: 60,611 (236.76 KB)

Trainable params: 60,611 (236.76 KB)

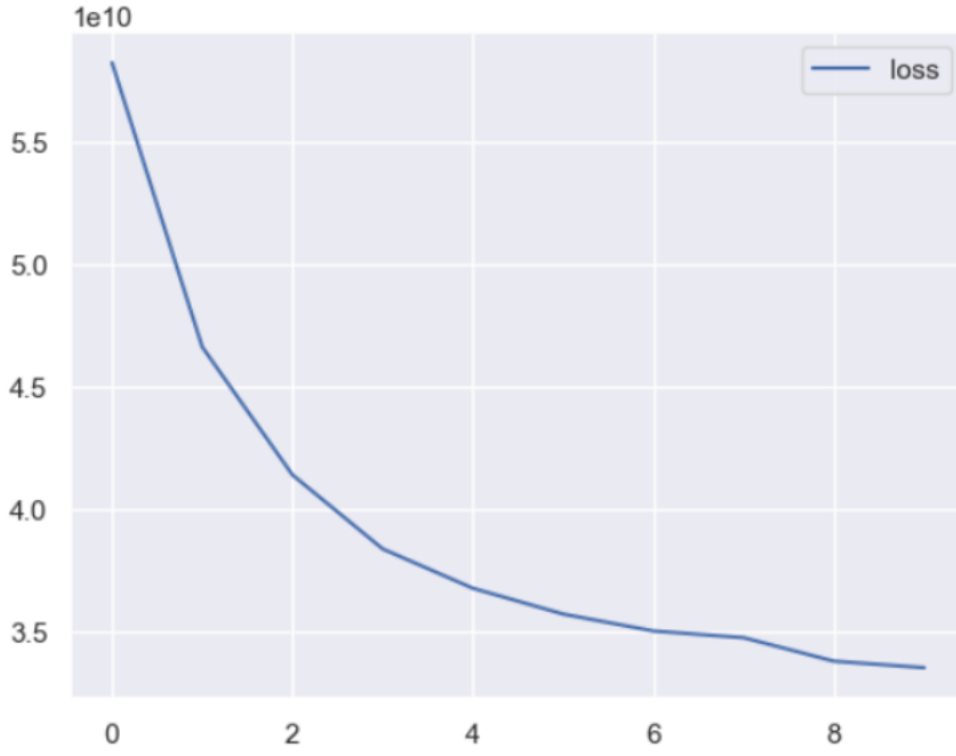
Non-trainable params: 0 (0.00 B)

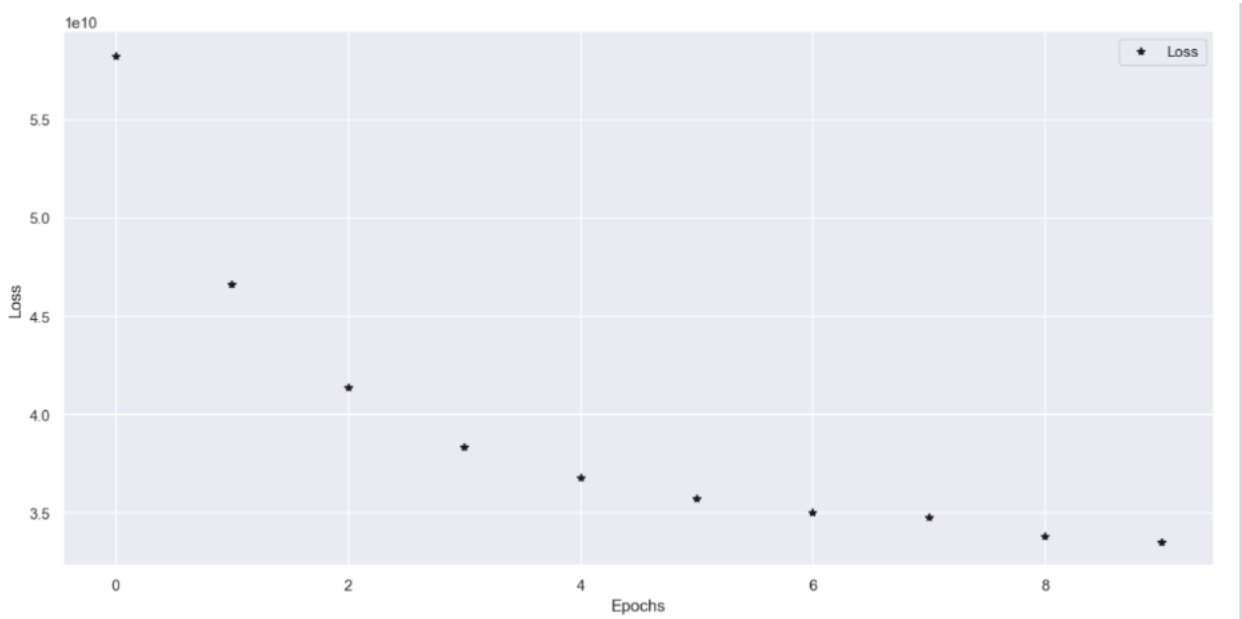
None

5]:
```
# ! add your chosen optimiser !

model.compile(optimizer='Nadam', loss='mse')
```

]: <Axes: >





LAB : 5

[55]:
```
model = Sequential()

# CONVOLUTIONAL LAYER
model.add(Conv2D(filters=32, kernel_size=(4,4),input_shape=(32, 32, 3), activation='relu',))
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

# Add more convolution and pooling layers with your choice of filters, strides, and activation function
# ...
# ...
model.add(Conv2D(filters=44, kernel_size=(4,4), activation='relu',))
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

model.add(Conv2D(filters=56, kernel_size=(4,4), activation='relu',))
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

model.add(Flatten())

# Add a few dense layers.
# ...
# ...
model.add(Dense(70, activation='relu'))
model.add(Dense(60, activation='relu'))
model.add(Dense(40, activation='relu'))

#LAST LAYER IS THE CLASSIFIER, THUS 10 POSSIBLE CLASSES
model.add(Dense(10, activation='softmax'))
```

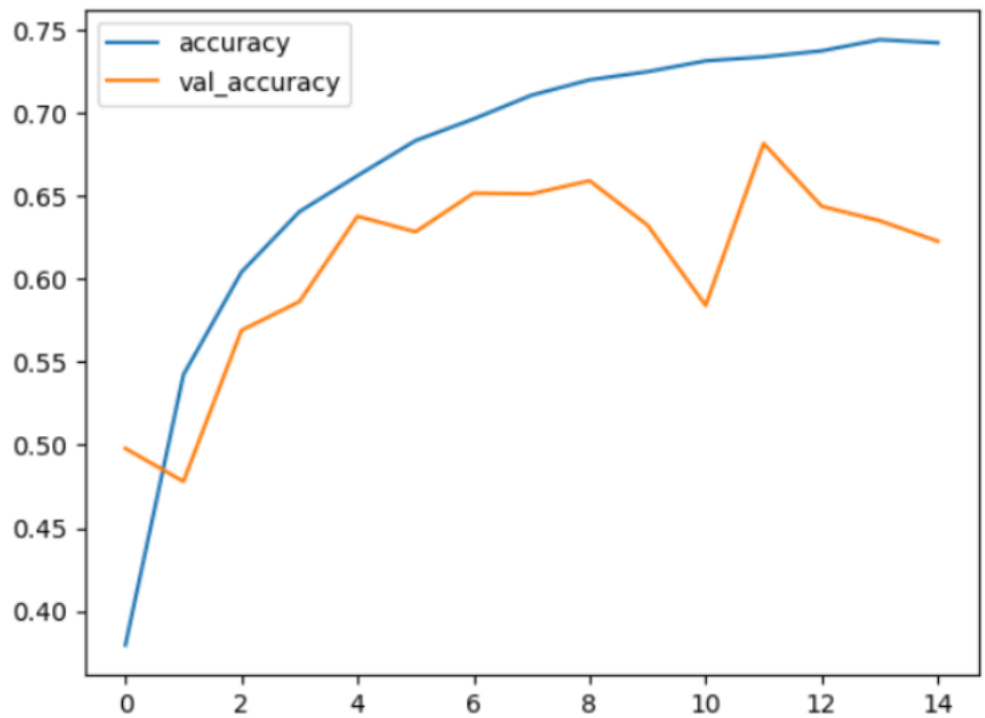
```
[56]: model.summary()
```

Model: "sequential_1"

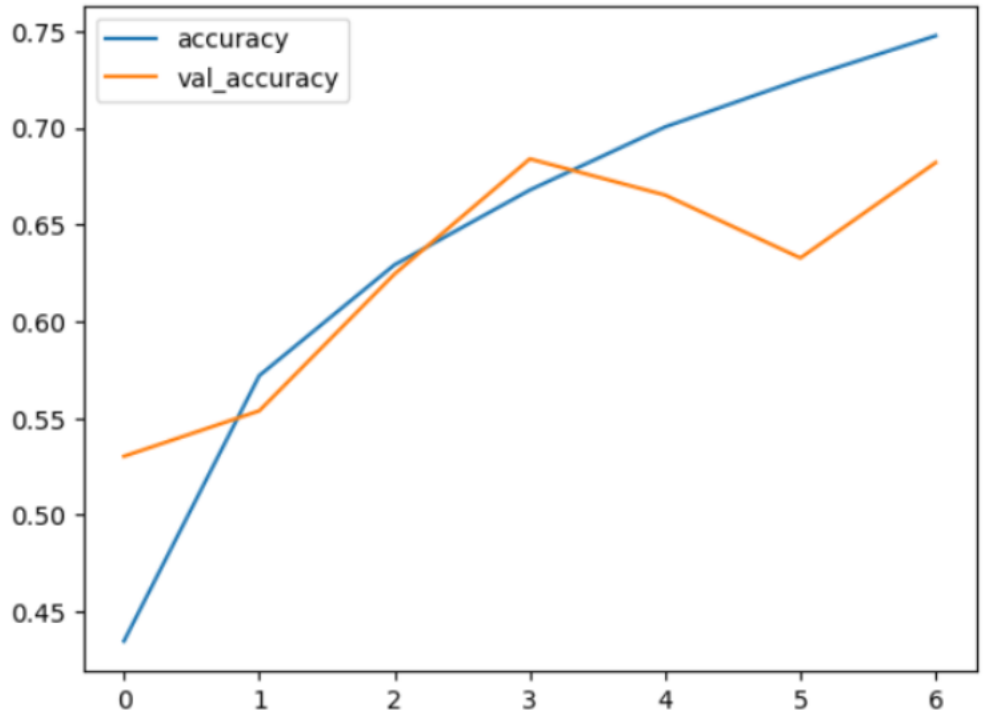
Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 29, 29, 32)	1,568
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 11, 11, 44)	22,572
max_pooling2d_4 (MaxPooling2D)	(None, 5, 5, 44)	0
conv2d_5 (Conv2D)	(None, 2, 2, 56)	39,480
max_pooling2d_5 (MaxPooling2D)	(None, 1, 1, 56)	0
flatten_1 (Flatten)	(None, 56)	0
dense_4 (Dense)	(None, 70)	3,990
dense_5 (Dense)	(None, 60)	4,260
dense_6 (Dense)	(None, 40)	2,440
dense_7 (Dense)	(None, 10)	410

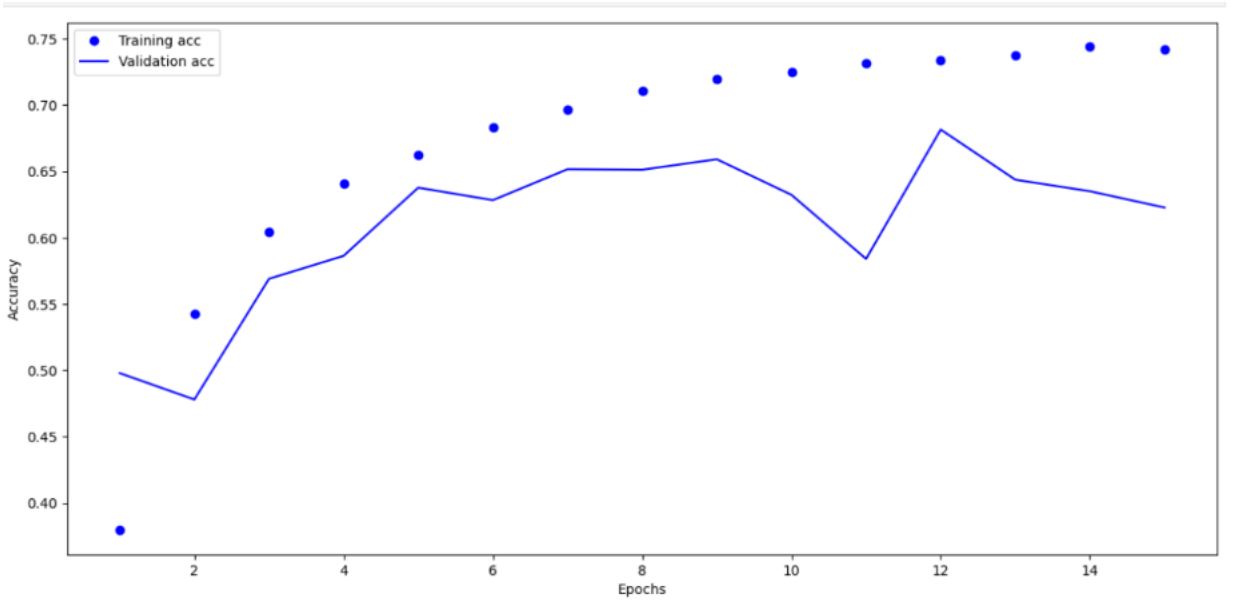
Total params: 74,720 (291.88 KB)

Trainable params: 74,720 (291.88 KB)

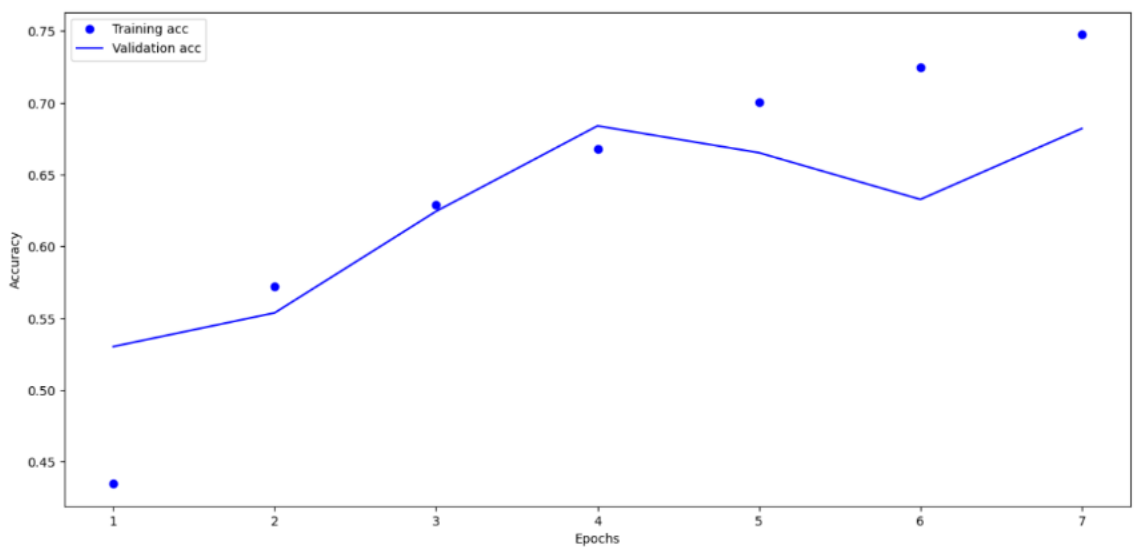


[91]: <Axes: >





[96]:



LAB : 6

```
model =Sequential()
model.add(GRU(150,activation='relu',return_sequences='True',input_shape=(length , n_features)))
model.add(GRU(80,activation='relu'))
model.add(Dense(1))
```

[31]:
```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 12, 150)	68,850
gru_1 (GRU)	(None, 80)	55,680
dense (Dense)	(None, 1)	81

Total params: 124,611 (486.76 KB)

Trainable params: 124,611 (486.76 KB)

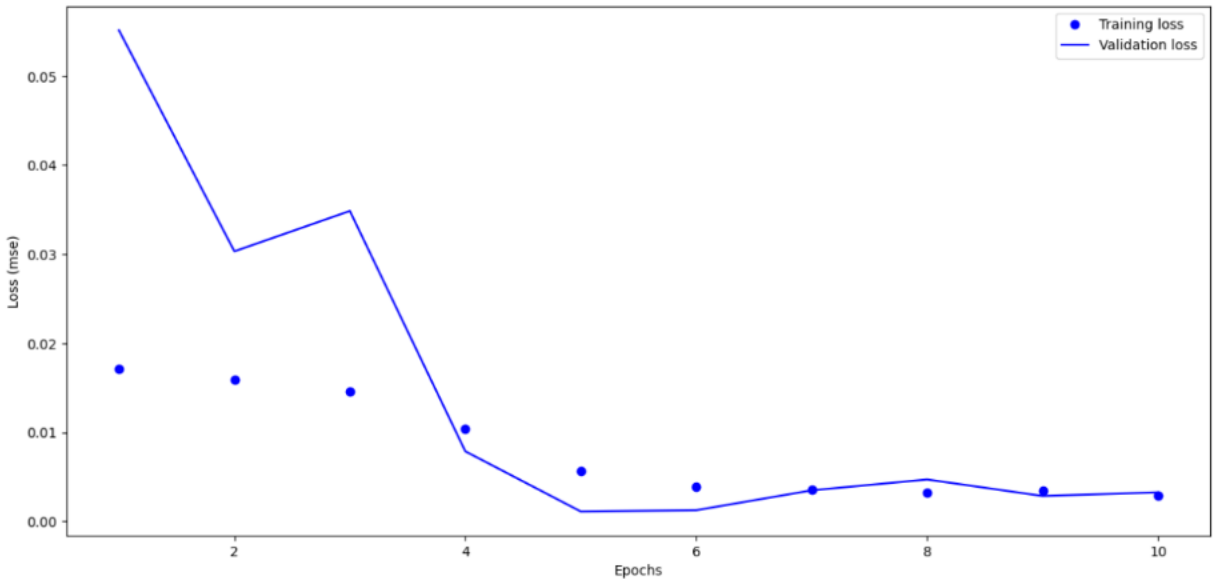
Non-trainable params: 0 (0.00 B)

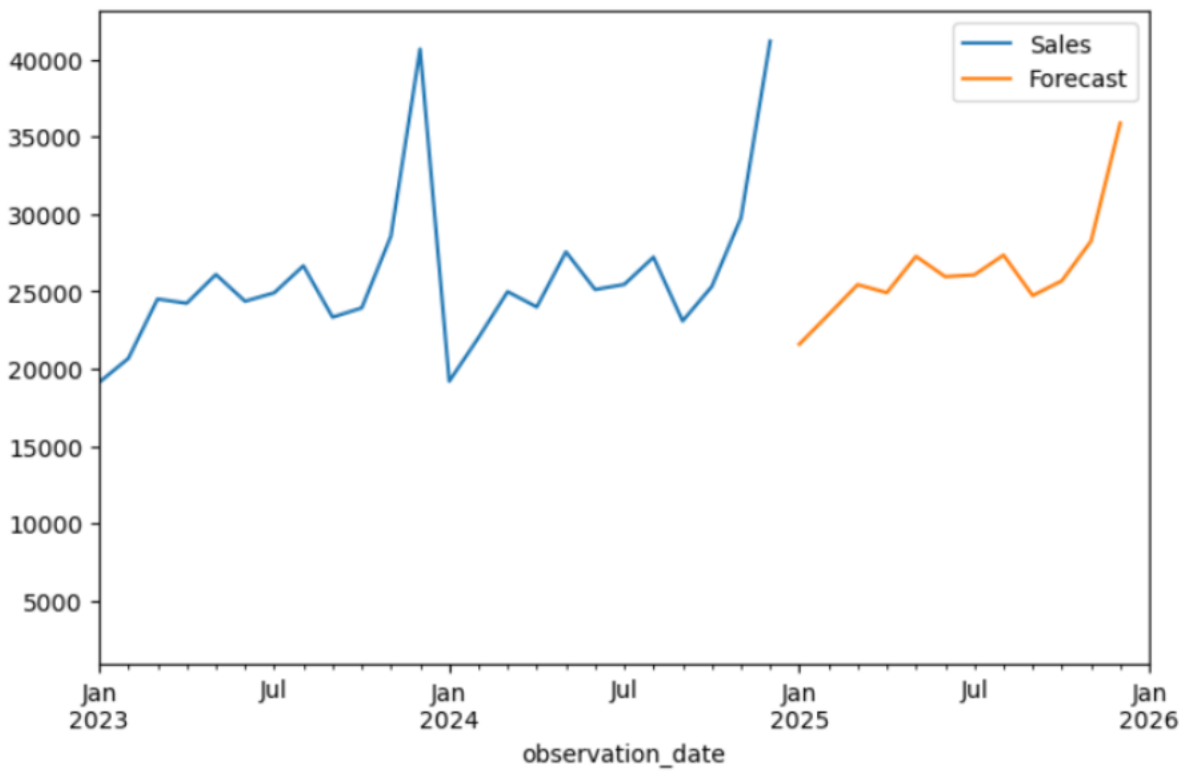
```
LAST_DIGIT_IN_YOUR_SID =21

early_stop = EarlyStopping(monitor='val_loss',patience=LAST_DIGIT_IN_YOUR_SID)
```

[36]:
```
history=model.fit(generator,epochs=10,validation_data=validation_generator,callbacks=[early_stop])
```

```
Epoch 1/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - loss: 0.0221 - val_loss: 0.0787
Epoch 2/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0159 - val_loss: 0.0543
Epoch 3/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - loss: 0.0148 - val_loss: 0.0366
Epoch 4/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0141 - val_loss: 0.0246
Epoch 5/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0094 - val_loss: 0.0035
Epoch 6/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0049 - val_loss: 2.6725e-04
Epoch 7/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0038 - val_loss: 0.0055
Epoch 8/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0039 - val_loss: 5.2924e-04
Epoch 9/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - loss: 0.0038 - val_loss: 8.2010e-04
Epoch 10/10
366/366 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.0035 - val_loss: 0.0041
```


Training loss
Validation loss
Loss (mse)
0.05
0.04
0.03
0.02
0.01
0.00
2
4
6
8
10
Epochs


Sales
Forecast
40000
35000
30000
25000
20000
15000
10000
5000
Jan 2023
Jul
Jan 2024
Jul
Jan 2025
Jul
Jan 2026
observation_date